SPSE4: A Multi-Relation Graph-Based Planning-Domain Editor for SWI-Prolog

System Description and Design Rationale for v0.1

Andrew Dougherty
FRDCSA / FRKCSA Project
adougher9@yahoo.com

May 2, 2026

Abstract

SPSE4 is a multi-relation graph-based planning-domain editor for SWI-Prolog, implemented as five composable Prolog packs totaling approximately 3,900 lines with 79 unit tests. It succeeds SPSE2 (2011, Perl/Tk) and SPSE2-Formalog (SWI-Prolog bridge) by consolidating the task ontology, temporal reasoning, planning-language I/O, scheduling, and reactive execution into a single process with a single data model, rooted in microtheories rather than Perl UniLang round-trips. The system integrates Allen's interval algebra with CLP(FD) for schedule feasibility, persistent microtheories with per-context access control and audit, a full PDDL/HDDL parser and emitter with a planner-capability match function, and a real-datetime scheduler with critical-path analysis and broadcast-based reactive triggers. This report describes the v0.1 architecture, the pack-by-pack decomposition, the key design decisions with their rationale, a worked end-to-end example driving the FRDCSA autopackager's priority queue, and the roadmap to v0.2.

Draft. This document is a working draft under active revision. Technical descriptions reflect the v0.1 codebase as of the date above, but prose, citations, and architectural framing may contain inaccuracies and are subject to change. Please consult the GitHub repository at https://github.com/aindilis/spse4 for the current source and the latest revision of this document. Feedback is welcome.

Contents

1 Introduction

The Shared Priority System Editor v2 (SPSE2) was described in the 2011 FRDCSA technical report *Temporal Planning and Inferencing for Personal Task Management with SPSE2* [Dougherty, 2011]. Its architecture tied together a Perl/Tk GUI, the FRDCSA UniLang messaging layer, the FreeKBS2 knowledge-base server, the Verber PDDL translator, and the LPG-TD planner. A companion module, SPSE2-Formalog, bridged portions of that world to SWI-Prolog so that knowledge-engineering and inference could happen where the Prolog infrastructure already was.

Over fifteen years of use, two things became clear. First, the round-trip architecture—Perl emitting UniLang messages to a FreeKBS2 image, Prolog pulling facts by *kquery* and flattening them into `holds/2`, Verber translating back out to PDDL, and a planner returning a plan that got re-read into the GUI—was a source of both latency and fragility. Second, the core data model (typed goals, typed binary edges, per-context assertion) was sound and worth preserving. What needed rebuilding was the infrastructure around it, not the ontology.

SPSE4 is that rebuild. It is not a port of SPSE2 to Prolog; it is a first-principles redesign of a planning-domain editor as a set of composable Prolog packs, any of which can be used standalone. The target audience is threefold: (1) the author's own daily planning and scheduling needs via the Free Life Planner (FLP); (2) the FRDCSA [Dougherty, 2026] subsystem ecosystem, including the autopackager's work queue; and (3) the wider SWI-Prolog and AI-planning communities, for whom `pack-allen`, `pack-mt-store`, and `pack-pddl` solve problems with no other clean SWI-Prolog solutions.

This v0.1 release ships the five foundational packs plus a live demonstration seeded with the autopackager's actual priority queue. It does not ship the collaboration server, the browser client, or the Emacs mode; those belong to v0.2, which is sketched in Section 9.

2 Background: What SPSE2 Was, and What Needed Rebuilding

The SPSE2 technical report was prepared as a submission to the ICAPS 2011 Workshop on Scheduling and Planning Applications but was not accepted for publication; it has remained available as a project document on the FRDCSA website. It describes a system with three conceptually separate layers entangled through the FRDCSA middleware:

1. An *ontology* of goals as graph nodes, with unary predicates marking status (`complete`, `cancelled`, `deleted`, `showstopper`, `ridiculous`, `obsoleted`, `rejected`, `skipped`) and binary predicates marking structural relations (`depends`, `provides`, `eases`, `prefer`). Per-goal metadata included `has-NL` (natural-language label), `has-source`, `asserter`, `start-date`/`end-date`, `costs`, `earns`, `severity`, `event-duration`, `has-formalization`, `has-parent`, `involves`, `ethicality-concern`, and `hard-time-constraints`.
2. A *PDDL bridge*: SPSE2 → Verber → PDDL → LPG-TD → plan, with calendar events (ICS, Google Calendar) translated into PDDL 2.2 timed initial literals and a durative `Complete` action gated by an `over all ((not has-time-constraints ?e) or possible(?e))` condition.
3. A *context system*: everything lived inside `holds(Context, Fact)`, with the contexts serving as microtheories in the Cyc sense—per-domain scopes the user could switch between.

The SPSE2-Formalog layer read contexts via shelling out to the Perl `spse2` binary, pulled facts via `kquery` into `holds/2` assertions, and exposed accessors like `goal/1`, `depends/2`, `has-NL/2` that

flattened the context prefix [Guha and Lenat, 1990]. The actual planning, GUI, and calendar logic remained in Perl/Tk and Emacs Lisp.

Three properties of that architecture motivated the rebuild:

- *No persistence beyond the Perl image.* The goal graph lived in a running Perl process's memory, re-asserted from dumps on startup. Concurrent access was impossible; the image was the bottleneck.
- *Scheduling round-trips had to go through PDDL.* Even simple "when can this happen given these constraints" queries involved PDDL generation and an external planner. This made sense in 2011 when Prolog's CLP(FD) was less mature; in 2026 it is unnecessary overhead for common cases.
- *The context system was ad-hoc.* `holds/2` worked, but there was no ACL, no audit trail, and no genuine inheritance in the Cyc `genlMt` sense.

SPSE4 addresses all three by (1) persisting microtheories through a dedicated store pack, (2) providing native CLP(FD)-based scheduling as the common case while keeping PDDL emission available for the hard cases, and (3) making microtheories first-class objects with owners, visibility, inheritance, and audit.

3 Design Rationale: Six Decisions That Shaped v0.1

Before the architecture is described, six design decisions taken during the v0.1 design session are worth stating explicitly, because they determined what v0.1 *is* and what it deliberately *is not*.

3.1 Native Prolog everywhere, no Perl round-trips

The entire SPSE4 runtime is SWI-Prolog. The SPSE2 legacy dump format is supported as an *importer*, not as a live bridge. A Perl installation is not required to use SPSE4. This decision follows from the consolidation argument in §2: the main pain in SPSE2 was the seams between components, and the simplest way to eliminate seams is to eliminate components.

3.2 Microtheories as first-class, persistent, ACL-aware objects

The `holds(Context, Fact)` pattern from SPSE2 is replaced by `ist(Mt, Fact)` in `pack-mt-store` (naming follows Guha's CycL convention [Guha and Lenat, 1990]), with `genlMt(Sub, Super)` for inheritance, per-microtheory owner and visibility, and an append-only audit log of every assertion and retraction. This is strictly more structure than SPSE2 provided, but the extra machinery is what enables the collaboration server planned for v0.2: you cannot broadcast diffs safely without knowing who asserted what, when, under whose authority.

3.3 Scheduling is CLP(FD)-native; PDDL is an exporter

Allen's thirteen interval relations [Allen, 1983] are implemented directly over CLP(FD) [Triska, 2012] finite-domain variables in `pack-allen`. Common scheduling questions—"is this set of relations consistent?", "find concrete start/end times for these intervals"—are answered by the constraint solver without any PDDL round trip. `pack-pddl` emits (and parses) PDDL when the user explicitly wants to hand a domain and problem to an external planner, but this is never on the critical path for daily use.

3.4 Native Prolog behavior-tree executor, not BT.CPP

The reactive execution layer in v0.2 will be a native Prolog behavior-tree interpreter, not a binding to BT.CPP. Rationale: the FLP use case is cognitive-prosthetic task execution, not robotics; BT.CPP's runtime properties (real-time guarantees, C++ callable leaves) are not needed, and embedding Prolog into a BT.CPP tick loop is architecturally awkward. If a robotics use case ever emerges, BT.CPP XML is a one-way exporter target—no runtime dependency on the other direction.

3.5 CycL is emitted as text, not called live

There is no live SWI-Prolog ↔ Cyc client in v0.1. `pack-spse4-core` emits `.cycl` files using the vocabulary from the SPSE2 Logical Operators work (`#$isa`, `#$SPSE2Mt`, `(am '(...) Mt)` syntax). When a working Prolog-to-Cyc bridge exists (Doug Miles's PrologCyc or another collaborator's reimplementation), it can consume these files. The system builds against a stable text format and does not depend on a live Cyc image.

3.6 Each pack is standalone-useful

The five packs are designed so that each one is worth installing by itself. `pack-allen` is a complete Allen interval algebra implementation for anyone doing temporal reasoning in SWI-Prolog; `pack-mt-store` is a persistent microtheory store for anyone doing KR; `pack-pddl` is a PDDL parser and emitter for anyone working with AI planners. The SPSE4-specific packs sit on top of these, but the foundation packs do not know or care about SPSE4. This increases the blast radius of bug reports (anyone using `pack-pddl` may find bugs in it, which benefits SPSE4) and opens a straightforward contribution path for the wider community.

4 Architecture Overview

The v0.1 architecture is a three-layer dependency graph:

```
pack-spse4-scheduler    (datetimes, critical path, reactive triggers)
  ↓ depends on ↓
pack-spse4-core    (task ontology, edges, projections, SPSE2 import)
  ↓ depends on ↓
pack-mt-store   pack-allen   pack-pddl
```

- The *foundation layer* (`pack-allen`, `pack-mt-store`, `pack-pddl`) has no SPSE4-specific dependencies. Each pack is independently installable and useful to the wider community.
- The *core layer* (`pack-spse4-core`) defines the task ontology and depends on `pack-mt-store` (for microtheory storage) and `pack-allen` (for temporal relations on task intervals). It also contains the SPSE2 legacy importer and the projection queries that filter the graph by relation kind, status, or microtheory membership.
- The *scheduler layer* (`pack-spse4-scheduler`) adds real-datetime scheduling, critical-path analysis, and broadcast-based reactive triggers, depending on `pack-spse4-core` and `pack-allen`.

All packs follow the Prolog coding conventions of Covington et al. [2012] and Bratko [2011]: `:- module/2` on every file, PlDoc docstrings on every exported predicate, PlUnit test files colocated in `t/`, strings for all user-visible text, and atoms for identifiers and enumerated values. These conventions are also consistent with the broader SWI-Prolog ecosystem [Wielemaker et al., 2012].

5 The Five Packs

5.1 `pack-allen`: Allen Interval Algebra + CLP(FD) Scheduler

Approximately 200 lines, 9 tests. Implements the thirteen primitive binary relations between closed intervals on a linear timeline [Allen, 1983]: `before`, `meets`, `overlaps`, `starts`, `during`, `finishes`, `equals`, and their inverses `after`, `met_by`, `overlapped_by`, `started_by`, `contains`, `finished_by`.

The module exports:

- `allen_relation/1`, `allen_inverse/2`, `allen_compose/3`—the qualitative layer: relation enumeration, inverse lookup, and the composition table (what relation holds between A and C, given A/B and B/C).
- `interval/3`, `constrain_interval/3`—interval declaration with CLP(FD) start and end variables, and constraint posting by qualitative relation.
- `schedule/3`—given a list of intervals and a list of qualitative constraints, find concrete integer start/end times satisfying all constraints.
- `relation_implies/2`—strict-to-loose reasoning (e.g. `starts` implies "not after").

A typical call:

```
?- schedule([a, b, c],
            [ constraint(a, before, b),
              constraint(b, overlaps, c) ],
            Solution).
```

returns concrete integer timelines for `a`, `b`, `c` that respect both constraints, or fails if the constraint set is inconsistent. The `schedule/3` predicate is the bridge between qualitative Allen reasoning and concrete integer timelines; `pack-spse4-scheduler` layers real datetime arithmetic on top by treating the integers as Unix-epoch seconds.

Why this matters standalone. There is no other clean Allen-algebra implementation for SWI-Prolog. The composition table alone is a reusable contribution; wiring it to CLP(FD) makes it directly useful for scheduling.

5.2 `pack-mt-store`: Persistent Microtheory Store

Approximately 300 lines, 10 tests. Implements a microtheory store with `ist(Mt, Fact)`, `genlMt(Sub, Super)`, `specialization/2`, per-microtheory owner and visibility metadata, and an append-only audit log. The name and vocabulary follow the CycL conventions of Guha and Lenat [1990] so the code reads naturally to anyone familiar with Cyc.

Core predicates: `mt_create/2` (create a microtheory with owner and visibility options); `mt_specialize/2` (declare a `genlMt` relation, i.e. inheritance); `mt_assert/2`, `mt_retract/2` (assertion and retraction, audit-logged); `ist/2` (query facts local to a microtheory); `ist_inherited/2` (query facts visible through the `genlMt` lattice); and `mt_audit/2` (stream the audit log for a microtheory).

Example:

```
?- mt_create(general_kb, [owner=andrew, visibility=public]).
?- mt_create(medical_kb, [owner=andrew]).
?- mt_specialize(medical_kb, general_kb).
?- mt_assert(general_kb, organism(andrew, human)).
?- mt_assert(medical_kb, takes(andrew, vitaminD)).
?- ist_inherited(medical_kb, Fact).
Fact = takes(andrew, vitaminD) ;
Fact = organism(andrew, human).
```

Vocabulary mapping:

SPSE2	SPSE4 `pack-mt-store`
`holds(Context, Fact)`	`ist(Context, Fact)`
context inheritance (ad-hoc)	`specialization/2`, `genlMt/2`

Cyc	SPSE4 `pack-mt-store`
`(ist Mt Fact)`	`ist(Mt, Fact)`
`(genlMt Sub Super)`	`genlMt(Sub, Super)`

v0.1 stores microtheories in-memory with persistence hooks in place but not yet wired; v0.2 will add the MySQL backend via the planned `prolog-mysql-store` integration for true multi-process persistence.

5.3 `pack-spse4-core`: Task Ontology and Graph Operations

Approximately 450 lines, 13 tests. Defines the core data model of SPSE4: a *priority system* is a labelled directed multigraph stored in one or more microtheories, with tasks as nodes and typed binary relations as edges.

All user-visible text (NL labels, descriptions, notes) is stored as **strings**; internal identifiers, predicate names, relation kinds, and enumerated values are **atoms**. This follows Covington's coding conventions.

Task properties:

Key	Value type
`has_nl`	string — natural-language label
`description`	string — longer text
`task_kind`	atom — `primitive` | `compound` | `recurring` | `ongoing` | `milestone`
`overlap_class`	atom — `exclusive` | `foregroundable` | `backgroundable` | `ambient` | `batched`
`status`	atom — `open` | `in_progress` | `completed` | `cancelled` | `deleted` | `skipped` | `obsoleted` | `rejected` | `showstopper` | `ridiculous` | `habitual`
`duration`	term — `fixed(N)` | `range(Lo,Hi)` | `distribution(K,Ps)` | `effort(Total)`
`earliest_start`	datetime term
`latest_finish`	datetime term
`needs_resource`	atom — may repeat
`costs`, `earns`	term — e.g. `dollars(30)`
`recurrence`	string — RFC 5545 RRULE
`triggers_when`	callable goal
`source`	arbitrary term (e.g. `spse2(Ctx)` to track provenance)

Edges carry an explicit `relation_kind` discriminator so the same pair of tasks can stand in multiple relations simultaneously:

- `depends`—A must complete before B can start (causal prerequisite)
- `provides`, `eases`, `prefer`—the SPSE2 relations, preserved
- `attacks`, `supports`—argumentation relations for contingent-plan reasoning
- `subsumes`—HTN-style abstraction
- `temporal/Allen-relation`—qualitative temporal constraints

Graph projection is composable. The projection predicate takes a filter expression built from `kind/1`, `status/1`, `in_mt/1`, `and/2`, `or/2`, `not/1`, and returns the induced subgraph:

```
% Project the dependency subgraph of medical_kb,
% excluding completed tasks:
?- project_graph(medical_kb,
                 and(kind(depends), not(status(completed))),
                 proj(Nodes, Edges)).
```

The SPSE2 legacy importer reads `holds/2` dumps and maps them to the new ontology:

```
?- import_spse2_file('~/spse2-dumps/pse.holds', legacy_pse).
```

The importer translates `holds(Ctx, goal('entry-fn'(P,I)))` to a task, `holds(Ctx, 'has-NL'(..., NL))` to the `has_nl` string, `holds(Ctx, depends(A,B))` to an `edge(A, depends, B)`, `holds(Ctx, complete(...))` to `status=completed`, and the other SPSE2 statuses, `costs`, `earns`, etc. analogously.

5.4 `pack-pddl`: PDDL/HDDL Parser and Emitter

Approximately 560 lines, 20 tests. A full DCG-based parser and emitter for PDDL 1.2, 2.1 [Fox and Long, 2003], 2.2 [Edelkamp and Hoffmann, 2004], 3.0, 3.1, and HDDL, together with a planner-capability table and a `required_features/2` utility that inspects a parsed domain for the PDDL features it uses.

Key predicates: `parse_pddl/2` (parse a PDDL file into a structured Prolog term); `emit_pddl/2` (emit a structured term as PDDL text); `required_features/2` (inspect a parsed domain, return the set of PDDL features it uses, such as `:typing`, `:durative-actions`, `:timed-initial-literals`); `eligible_planners/2` (given a feature set, return planners capable of handling it, from the capability table).

The round-trip property *parse* ∘ *emit* ∘ *parse* ≡ *parse* is verified by a dedicated test. The planner table covers the common temporal and HTN planners and is data-driven—adding a new planner is a single `planner_capability/2` fact.

Why this matters standalone. No other SWI-Prolog pack provides a complete PDDL parser across the language versions. The feature-detection utility is particularly useful for the autopackager's workflow of deciding which planner to hand a given domain to.

5.5 `pack-spse4-scheduler`: Real-Datetime Scheduling and Reactive Triggers

Approximately 450 lines, 15 tests. Extends `pack-allen`'s integer-timeline scheduler to real calendar time, adds deadline propagation and critical-path analysis, and wires `when/2`-driven reactive triggers that fire when a task's dependencies become satisfied.

Real datetimes use SWI-Prolog's built-in `library(date)` (Unix-epoch seconds as the internal representation) with `format_time/3` for display. No external date library is required; callers who prefer the `julian` pack can still use its values as inputs since any integer seconds-since-epoch is accepted.

Core capabilities:

- `schedule_tasks/3`—schedule all tasks in a microtheory respecting asserted temporal relations, durations, earliest_starts, and latest_finishes.
- `critical_path/3`—compute the critical path from ready tasks to a goal task, returning the path along with its length in seconds.
- `propagate_deadlines/2`—given a `latest_finish` on a downstream task and known durations of its upstream dependencies, compute the implicit `latest_finish` of the upstream tasks (classical PERT backward pass).
- `task_earliest_start/3`, `task_latest_finish/3`, `task_slack/3`—point-query interface for individual task timing and slack (latest-start minus earliest-start).
- `on_dependencies_complete/3`—register a goal that should fire when a given task's dependencies all reach `status=completed`.
- `trigger_ready_tasks/1`—scan a microtheory and broadcast `ready_task_event(Mt, TaskId)` for every task whose dependencies are now satisfied.

The reactive layer uses `library(broadcast)` rather than `when/2` directly, because `when/2`'s coroutining semantics tie the trigger to a specific query scope; broadcast gives the server-authoritative fanout the v0.2 collaboration layer will need (see Lager and Wielemaker [2014] for the Pengines transport this enables).

6 The Autopackager Demo: An End-to-End Walkthrough

The demo at `autopackager_demo.pl` seeds SPSE4 with eleven tasks drawn from the FRDCSA autopackager's actual priority queue—package `eprover`, `peleus`, `vampire`, `acl2`, `freekbs2`, submit to mentors, prepare the FLP public release, and several intermediate milestones. It then exercises every major SPSE4 capability on real data.

6.1 Seeding the microtheory

The demo opens by creating a microtheory and asserting eleven tasks with their dependencies, durations, statuses, and declared earliest_starts. Three are already `completed` (`pkg_eprover`, `pkg_peleus`, `pkg_superlu`), one is `in_progress` (`pkg_vampire`), and the rest are `open`.

```
Seeded autopackager microtheory with 11 tasks.
```

6.2 Identifying ready tasks

The demo then projects the dependency graph and identifies which tasks have all prerequisites satisfied, producing the list of tasks that could be started immediately.

6.3 Critical path to the FLP public release

`critical_path/3` is called with `pkg_flp_public_release` as the goal, and returns the critical path along with its cumulative duration. On the seeded data this path runs through `pkg_vampire` (the current in-progress task), which is the actual bottleneck in the real queue.

6.4 Schedule projection with real dates

`schedule_tasks/3` runs the full CLP(FD) schedule against the asserted durations and earliest_starts. For the tasks with declared start dates (`pkg_vampire` at April 22, 2026), the scheduler places them on their declared dates; for tasks without declared starts, it schedules them starting from "today" (the demo's run date). Dependencies are honored: `submit_autopkg_to_mentors` is placed after `pkg_vampire` completes.

Representative output (edited for brevity):

```
pkg_vampire        : 2026-04-22 -> 2026-04-25  (3 days)
submit_mentors     : 2026-04-25 -> 2026-04-26  (1 day, waits on vampire)
pkg_acl2           : 2026-04-22 -> 2026-04-24
pkg_freekbs2       : 2026-04-22 -> 2026-04-23
pkg_flp_release    : 2026-05-04 -> 2026-05-04  (milestone)
```

6.5 Readiness gate with blocker identification

A gate check on `submit_autopkg_to_mentors` reports:

```
  pkg_eprover: completed
  pkg_peleus: completed
  pkg_vampire: in_progress
  NOT YET -- 1 blocker(s) outstanding:
    pkg_vampire (in_progress)
```

The gate not only returns a boolean but names the specific blockers and their current statuses—this is the actual information a user needs to act on the gate failure.

6.6 Reactive trigger demonstration

Finally the demo registers a reactive trigger: "when `submit_autopkg_to_mentors` becomes ready, announce so." The trigger registers silently. The demo then simulates `pkg_vampire` completing (by updating its status to `completed` and calling `trigger_ready_tasks/1`), and the trigger fires automatically with the announcement.

```
Simulating vampire completion:
  *** AUTOMATIC: all prereqs complete, time to submit! ***
```

6.7 Why this demo matters

Every piece of the classical command-and-control observe-orient-decide-act loop is represented in this single end-to-end run: perception (task status queries), comprehension (dependency graph projection), projection (the schedule), decision (gate checks, critical-path identification), and action (reactive trigger fires on state change). This is real C2 reasoning on real operational data—not a toy. It is the first time SPSE4 has done something meaningful with the author's actual planning queue, and it validates the architecture: the five packs compose cleanly, the demo is ≈200 lines of application code, and the user-visible behavior is correct on the first full end-to-end run.

7 Running the System

7.1 Installation

Install SWI-Prolog ≥ 9.0.0. Then either (a) use the five packs as a meta-project from the SPSE4 source tree, or (b) install each pack individually via `pack_install/1` from its GitHub URL (planned for v0.2).

Meta-project use:

```
cd ~/Downloads/spse4
swipl run_all_tests.pl          # expected: 79 tests green, no warnings
swipl autopackager_demo.pl      # expected: end-to-end demo output
```

7.2 Loading individual packs from Prolog

```
?- use_module(library(allen)).
?- use_module(library(mt_store)).
?- use_module(library(spse4_core)).
?- use_module(library(pddl)).
?- use_module(library(spse4_scheduler)).
```

7.3 Per-pack test suites

Each pack ships a `t/*.plt` PlUnit file. Run with:

```
swipl -g "[t/allen], run_tests" -t halt pack-allen/prolog/allen.pl
```

8 Coding Conventions

The entire codebase follows a small set of explicit rules, consistent across all five packs:

- *Covington-Bratko style.* Lowercase-underscore predicate names, `:- module/2` on every source file, PlDoc docstrings on every exported predicate, PlUnit tests colocated as `t/<module>.plt`.
- *Strings for user-visible text.* Labels, descriptions, natural-language fragments, free-form notes—all SWI strings (`""` quoted).
- *Atoms for identifiers and enums.* Task IDs, relation kinds, status values, microtheory names, predicate names—all Prolog atoms (`''` or unquoted).
- *No side effects outside designated I/O modules.* Predicates that mutate the store are in `mt_store` and nowhere else.
- *Every exported predicate has both a ground-case test and a variable-binding test.* This catches the most common class of bug (a predicate that works for queries but fails when asked to generate bindings, or vice versa).

These rules came out of the SPSE4 v0.1 session and are recorded here because they are the main reason the codebase held together across twelve bug-fix iterations without structural refactoring.

9 Roadmap to v0.2

v0.1 delivers the foundational layer. The next release delivers the collaboration layer and two additional reasoning layers.

9.1 `pack-spse4-server`: Pengines HTTP/WebSocket Server

A SWI-Prolog Pengines server with basic authentication, per-microtheory access control enforcement, and `library(broadcast)` fanout so clients see diffs in near-real-time. Approximately 300 lines. Endpoints follow the standard Pengines API [Lager and Wielemaker, 2014] (`pengine/create`, `pengine/send`, `pengine/pull_response`, `pengine/destroy`) extended with an SPSE4-specific projection-filter endpoint that returns only the nodes and edges visible under a given filter expression.

9.2 `spse4-web`: Cytoscape.js Client

A browser client using Cytoscape.js for graph rendering and `pengines.js` for the Prolog connection. The projection UI exposes checkboxes for `relation_kind`, `in_mt`, and `status`; nodes are colored by status and by critical-path membership. Approximately 500 lines of JS/HTML, single page, no framework.

9.3 `spse4-mode.el`: Emacs Mode

An Emacs mode using `pengine.el` (already written) to provide the same projection operations from inside Emacs, so SPSE4 becomes directly editable from `org-mode` buffers without leaving the editor.

9.4 Additional reasoning layers

Three additional packs are planned for v0.2 or v0.3, each independently motivated:

- *Native Prolog behavior-tree executor.* Reactive execution of task decompositions, with tick-based evaluation and standard BT control flow (sequence, selector, parallel, decorator). No external runtime dependency.
- *Event Calculus layer (RTEC-inspired).* For temporal reasoning beyond Allen intervals, particularly for recurrence and for reasoning about what happened when; `pack-allen` answers "what qualitative relation holds", the Event Calculus layer will answer "what fluent holds at time t".
- *s(CASP) plan justification.* For every scheduled plan, produce a proof tree showing why each scheduling decision was forced; lets the user audit the scheduler's reasoning step-by-step.

9.5 Exports

- *CycL text emission.* Already sketched (see §3.5); v0.2 will finalize the emitter and add a round-trip test using a stable `.cycl` corpus.
- *RDF/Turtle export.* For federation with Koo5's work and for compatibility with external knowledge-graph tooling.
- *HTN SHOP-style meta-interpreter.* Task decomposition using the SHOP HTN style, with SPSE4's `subsumes` edges as the method-precondition source.
- *planutils adapter.* A thin adapter for the `planutils` polyglot-planner backend, so `emit_pddl` → `planutils` → parsed plan is a one-liner.

10 Discussion

Three properties of the v0.1 delivery are worth highlighting explicitly.

Iteration discipline. The session produced twelve tarballs (v0.1.0 → v0.1.12). Each tarball fixed exactly one bug, caught with one targeted diagnostic, with no structural overshoot. The bugs fell into categories that are well-documented SWI-Prolog gotchas—`assertz` copying attribute variables, `forall/2` undoing CLP(FD) bindings, choice-point warnings on tail-recursive helpers—and each fix was local. The root cause, in every case, was a subtle semantics mismatch between the programmer's model and the actual Prolog semantics; none of the bugs were design flaws. This is consistent with the "tight feedback loop on real data" methodology the FRDCSA project has consistently used.

Scope discipline. v0.1 is aggressively scoped. Things deliberately deferred include: the collaboration server, the browser client, the Event Calculus layer, the s(CASP) justification layer, live Cyc integration, and the MySQL backend for `mt_store`. Every deferral has a stated rationale. This is what allowed v0.1 to ship with 79 green tests and an end-to-end working demo in a single session.

Dogfooding from day one. The autopackager demo uses the author's actual packaging queue, not a toy. This matters for two reasons. First, it forces the user-visible ergonomics to be correct—there is no way to fake a realistic demo. Second, it means SPSE4 is a working tool for the author on the day of the v0.1 release, not a tool waiting for content to be useful. The FRDCSA project has consistently prioritized this kind of self-hosting ("the Free Life Planner must first plan its own development"), and SPSE4 v0.1 meets the bar.

Acknowledgments

SPSE4 builds on intellectual lineage from Allen [1983] on interval algebra, Guha and Lenat [1990] on microtheories and context logic, the PDDL community [Fox and Long, 2003, Edelkamp and Hoffmann, 2004], and the SWI-Prolog core team [Wielemaker et al., 2012]. The 2011 SPSE2 technical report provided the ontology. The v0.1 codebase was developed in a live pair-programming session between the author and Claude (Anthropic, Claude Opus 4.6 and 4.7) on April 22, 2026, with Claude producing the majority of the Prolog source and the author providing domain knowledge, architectural judgment, and test-failure diagnostics. The iteration pattern of "paste the error, fix one thing, re-run" proved remarkably effective across the twelve tarballs.

License

References

James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi:10.1145/182.358434.

Ivan Bratko. *Prolog Programming for Artificial Intelligence*. Pearson Education, 4th edition, 2011. ISBN 978-0-321-41746-6.

Michael A. Covington, Roberto Bagnara, Richard A. O'Keefe, Jan Wielemaker, and Simon Price. Coding guidelines for Prolog. *Theory and Practice of Logic Programming*, 12(6):889–927, 2012. doi:10.1017/S1471068411000391. Preprint at `https://arxiv.org/abs/0911.2899`.

Andrew Dougherty. Temporal planning and inferencing for personal task management with SPSE2. Technical report, FRDCSA Project, 2011. URL `https://frdcsa.org/visual-aid/pdf/Temporal-Planning-and-Inferencing-for-Personal-Task-Management-with-SPSE2.pdf`.

Andrew Dougherty. FRDCSA: Formalized research database – cluster, study and apply. `https://frdcsa.org/`, 2026. Project website, ongoing since 1998.

Stefan Edelkamp and Jörg Hoffmann. PDDL2.2: The language for the classical part of the 4th international planning competition. Technical Report 195, Albert-Ludwigs-Universität Freiburg, 2004.

Maria Fox and Derek Long. PDDL2.1: An extension to PDDL for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003. doi:10.1613/jair.1129.

Ramanathan V. Guha and Douglas B. Lenat. CYC: A midterm report. *AI Magazine*, 11(3):32–59, 1990. doi:10.1609/aimag.v11i3.842.

Torbjörn Lager and Jan Wielemaker. Pengines: Web logic programming made easy. *Theory and Practice of Logic Programming*, 14(4–5):539–552, 2014. doi:10.1017/S1471068414000192.

Markus Triska. The finite domain constraint solver of SWI-Prolog. In *Functional and Logic Programming – 11th International Symposium, FLOPS 2012*, volume 7294 of *Lecture Notes in Computer Science*, pages 307–316. Springer, 2012. doi:10.1007/978-3-642-29822-6_24.

Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. SWI-Prolog. *Theory and Practice of Logic Programming*, 12(1–2):67–96, 2012. doi:10.1017/S1471068411000494.